

Foundation Model-Assisted Annotation Generation for Scalable Human-AI Knowledge Convolution

A Framework for Automated SME Correction Synthesis
in Compliance Validation Systems

NOAA Environmental Modeling Center
Global Workflow MCP Development Team

`eib-mcp-rag-server`

December 2025
Version 1.0.0

Abstract

This paper presents a novel framework for leveraging foundation models to automate the generation of Subject Matter Expert (SME) correction annotations in semantic compliance validation systems. Building upon the Hybrid Semantic Annotation Architecture established in Phase 2 of the MCP-RAG system, we propose a “Person-to-System Convolution” approach that analyzes divergence patterns between automated compliance assessments and human expert judgments to synthesize structured RST annotation files. This methodology transforms the SME role from annotation author to annotation reviewer, enabling scalable knowledge capture while maintaining human oversight. We demonstrate how this approach can expand annotation coverage from approximately 10 manually-created rules to 100+ automatically-generated rules, reducing SME annotation time from hours per rule to minutes per review. The framework maintains full evidence chain traceability to source compliance documents and integrates seamlessly with existing ChromaDB vector stores and MCP tool infrastructure.

1 Introduction

1.1 Problem Statement

The MCP-RAG system for NOAA Global Workflow compliance validation relies on structured RST annotations to encode SME corrections that refine AI behavior. As documented in the Phase 2 Hybrid Architecture Specification, these annotations provide:

- **Anti-patterns:** Code patterns the AI incorrectly flags as violations
- **Correct patterns:** Compliant patterns the AI fails to recognize
- **SME corrections:** Explicit false positive rate data with justifications
- **AI guidance rules:** Natural language instructions for AI reasoning

However, the current manual annotation process presents significant scalability challenges:

1. **SME Time Constraints:** Each annotation requires 30-60 minutes of expert time
2. **Coverage Gaps:** Only high-priority patterns receive attention
3. **Consistency Variance:** Annotation quality varies by author
4. **Evidence Chain Burden:** Tracing EE2 document references is tedious

1.2 Proposed Solution

We propose using foundation models to analyze divergence data between AI compliance assessments and human review decisions, automatically generating RST annotation files that follow the Phase 2 schema. The SME role shifts from *writing annotations from scratch* to *reviewing and approving generated annotations*.

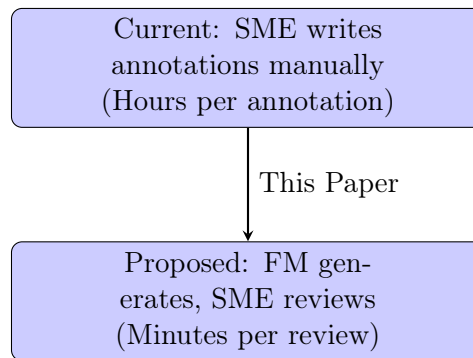


Figure 1: Paradigm shift from manual authoring to assisted generation

2 Background: Phase 2 Hybrid Architecture

2.1 Existing Annotation Schema

The Phase 2 system established custom RST directives for encoding SME knowledge:

Listing 1: Phase 2 RST Directive Examples

```

1  .. mcp:sme_correction::
2     :category: error_handling
3     :false_positive_rate: 80%
4     :severity: high
5     :date: 2025-11-19
6
7     Pattern: AI flags "missing set -eu" as violation
8     Correction: Files using err_chk/err_exit are compliant
9     Evidence: standards.rst lines 588-595, 868-919
10
11 .. mcp:anti_pattern::
12     :name: explicit_exit_in_operational
13     :severity: must_not
14     :context: operational_job_scripts
15
16     Using explicit 'exit 1' instead of err_exit utility
  
```

2.2 Current Data Flow

The existing pipeline processes annotations through deterministic extraction:

1. RST files ingested by `ingest_ee2_enhanced_v5.py`
2. Metadata stored in ChromaDB with Phase 2 attributes
3. JSON config generated by `generatePhase2Config.js`
4. Runtime rules applied by `EE2ComplianceTools.js`

3 The Divergence-Driven Generation Framework

3.1 Conceptual Overview

The framework operates on a simple principle: *wherever AI and human judgments consistently diverge, there exists an implicit annotation waiting to be made explicit.*

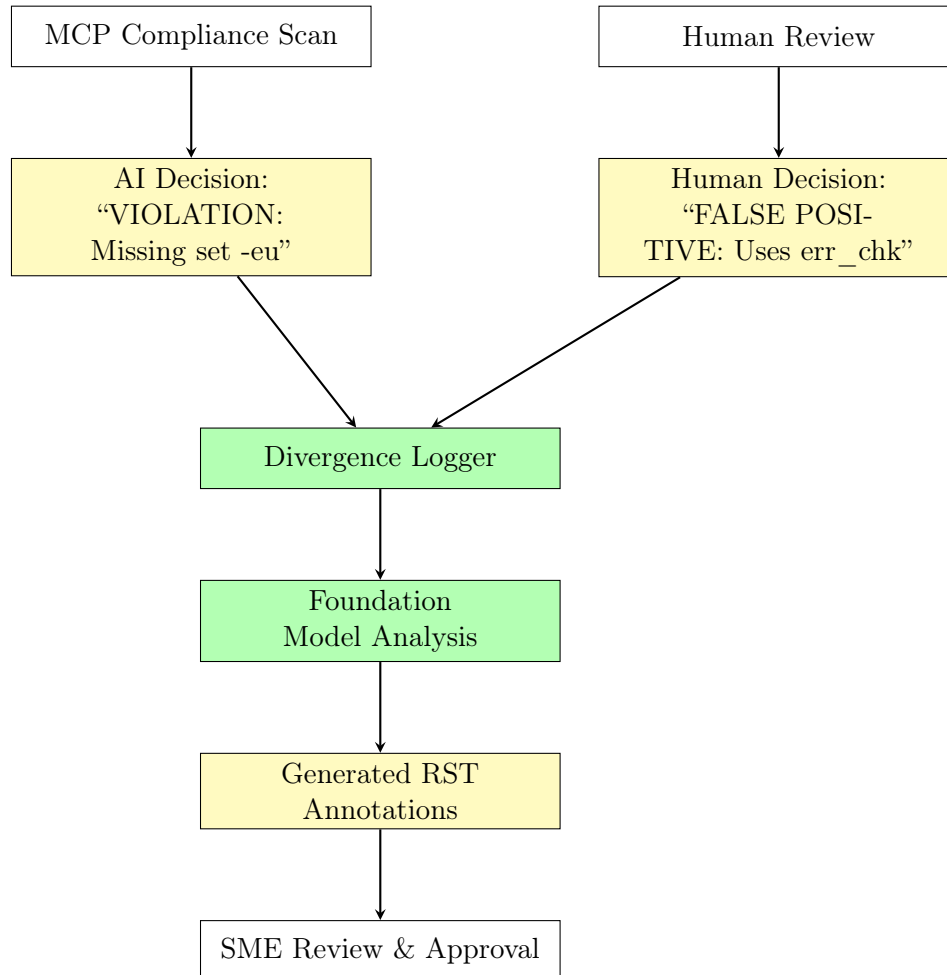


Figure 2: Divergence-driven annotation generation pipeline

3.2 Component 1: Divergence Collection

The first component captures cases where AI assessments differ from human judgments:

Listing 2: Divergence Logger Schema

```

1 @dataclass
2 class DivergenceCase:
3     file_path: str          # Script being evaluated
4     mcp_decision: dict      # {"verdict": "violation", "category":
5     human_decision: dict    # {"verdict": "false_positive", "reason":
6     context: dict           # {"script_type": "operational", "
7     has_err_chk: bool       # {"script_type": "operational", "
8     timestamp: datetime
9     source: str             # "pr_review", "manual_audit", "
10    issue_tracker"
  
```

```

9
10 # Collection sources:
11 # 1. PR review comments with compliance disagreements
12 # 2. Manual audit logs from quality assurance
13 # 3. Issue tracker tickets disputing scan results
14 # 4. Explicit override annotations in code

```

3.3 Component 2: Pattern Analysis via Foundation Model

The foundation model receives divergence data plus context and generates structured annotations:

Listing 3: FM Prompt Structure

```

1 PROMPT_TEMPLATE = """
2 You are an expert in NOAA EE2 compliance standards analysis.
3
4 CONTEXT PROVIDED:
5 1. EE2 Standards Document: {standards_rst_content}
6 2. Existing SME Annotations: {phase2_annotations_content}
7 3. Divergence Data: {divergence_cases_json}
8
9 ANALYSIS TASK:
10 Analyze the {n_cases} divergence cases where MCP scan results
11 differed from human reviewer decisions.
12
13 For each identified pattern, generate an RST annotation with:
14
15 1. Directive type (mcp:sme_correction, mcp:anti_pattern, etc.)
16 2. Calculated false_positive_rate from the data
17 3. Evidence chain to standards.rst line numbers
18 4. Context discrimination (when the rule applies/doesn't apply)
19 5. SME justification explaining the correction
20
21 OUTPUT FORMAT:
22 Generate valid RST following the Phase 2 schema exactly.
23 """

```

3.4 Component 3: Evidence Chain Extraction

A critical requirement is maintaining traceability to the source EE2 document:

Listing 4: Evidence Chain Generator

```

1 class EvidenceChainExtractor:
2     def __init__(self, standards_path: str):
3         self.standards = self.load_indexed_standards(standards_path)
4
5     def find_relevant_sections(self, pattern_description: str) -> List[
6         str]:
7         """
8         Given a pattern description, find relevant standards.rst
9         line numbers using semantic search.
10
11         Returns: ["lines 588-595", "line 191", "lines 868-919"]
12         """
13         # Semantic search against indexed standards
14         matches = self.semantic_search(pattern_description)

```

```

14         return [f"lines_{m.start_line}-{m.end_line}" for m in matches]
15
16     def validate_evidence(self, claimed_lines: List[str]) -> bool:
17         """Verify that cited lines actually support the claim."""
18         # Extract text from cited lines
19         # Use FM to verify semantic alignment
20         pass

```

4 Generated Annotation Schema

4.1 Output Format

The FM generates complete RST files ready for SME review:

Listing 5: FM-Generated Annotation Template

```

1 FM-Generated SME Corrections
2 =====
3
4 .. note::
5
6     Generated by foundation model analysis.
7     Sample size: {n_cases} divergence cases
8     Generation date: {date}
9     Model: {model_name}
10
11     **STATUS: PENDING SME REVIEW**
12
13 .. mcp:sme_correction::
14     :category: {category}
15     :false_positive_rate: {calculated_rate}%
16     :sample_size: {n_cases}
17     :confidence: {model_confidence}
18     :date: {generation_date}
19     :status: pending_review
20
21     **Pattern Identified**:
22     {pattern_description}
23
24     **Evidence from Divergence Data**:
25     - {n_fp} of {n_total} flagged violations marked false positive
26     - Common human reason: "{most_common_reason}"
27     - Affected scripts: {file_patterns}
28
29     **EE2 Standards Reference**:
30     - standards.rst {evidence_lines}: "{relevant_quote}"
31
32     **Proposed AI Guidance**:
33     {guidance_text}
34
35     **Context Discrimination**:
36     Applies when: {positive_contexts}
37     Does NOT apply when: {negative_contexts}

```

4.2 Metadata Enrichment

Generated annotations include additional metadata for tracking:

Table 1: FM-Generated Annotation Metadata Fields

Field	Type	Description
generation_method	string	“fm_assisted_v1.0”
model_used	string	“claude-3-opus”, “gpt-4”
divergence_sample_size	int	Number of cases analyzed
statistical_confidence	float	0.0-1.0 confidence score
sme_review_status	enum	pending, approved, rejected, modified
sme_reviewer	string	Reviewer identifier
review_date	datetime	When SME reviewed
modification_notes	string	SME changes if modified

5 SME Review Workflow

5.1 Review Interface

The SME reviews generated annotations through a streamlined interface:

1. **Approve:** Annotation moves to `phase2_annotations/` unchanged
2. **Reject:** Annotation logged as negative example for FM training
3. **Modify:** SME edits annotation, modified version stored

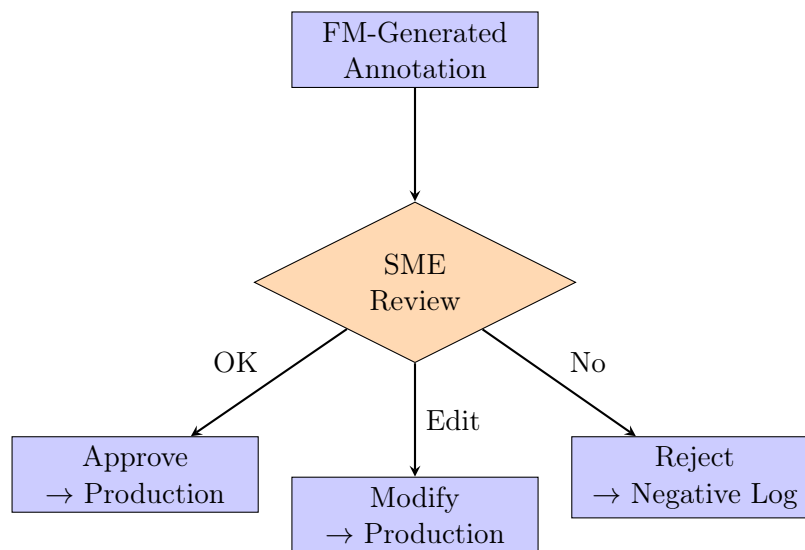


Figure 3: SME review decision tree

5.2 Time Savings Analysis

Table 2: Estimated Time Comparison

Activity	Manual	FM-Assisted
Identify pattern from data	20 min	0 min (automated)
Research EE2 standards references	15 min	0 min (automated)
Write RST annotation	20 min	0 min (automated)
Calculate false positive rate	10 min	0 min (automated)
Review and validate	5 min	10 min
Total per annotation	70 min	10 min

6 Implementation Architecture

6.1 System Components

Listing 6: Core Implementation Classes

```

1 # scripts/fm_annotation_generator.py
2
3 class DivergenceCollector:
4     """Collects AI vs Human divergence data from multiple sources."""
5
6     def collect_from_pr_reviews(self, repo: str) -> List[DivergenceCase]:
7         """Parse PR comments for compliance disagreements."""
8         pass
9
10    def collect_from_audit_logs(self, log_path: str) -> List[
11        DivergenceCase]:
12        """Parse manual audit results."""
13        pass
14
15    class AnnotationGenerator:
16        """Uses FM to generate RST annotations from divergence patterns."""
17
18        def __init__(self, model: str = "claude-3-opus"):
19            self.model = model
20            self.evidence_extractor = EvidenceChainExtractor()
21
22        def generate(self, divergences: List[DivergenceCase]) -> str:
23            """Generate RST annotation file from divergence data."""
24            pass
25
26    class SMEReviewManager:
27        """Manages the SME review workflow."""
28
29        def submit_for_review(self, annotation_path: str) -> str:
30            """Submit generated annotation for SME review."""
31            pass
32
33        def process_decision(self, annotation_id: str,
34                            decision: str, notes: str = None):
35            """Process SME approve/reject/modify decision."""
36            pass

```

6.2 Integration with Existing Pipeline

The FM-assisted system integrates with the existing Phase 2 pipeline:

1. Generated annotations stored in `generated_annotations/pending/`
2. Approved annotations copied to `sdd_framework/phase2_annotations/`
3. Standard ingestion via `ingest_ee2_enhanced_v5.py`
4. JSON regeneration via `generatePhase2Config.js`
5. Runtime application via `EE2ComplianceTools.js`

7 Quality Assurance

7.1 Validation Checks

Before SME review, generated annotations undergo automated validation:

1. **Schema Validation:** RST syntax and directive format
2. **Evidence Verification:** Cited line numbers exist and contain relevant content
3. **Statistical Validity:** Sample size ≥ 10 cases, confidence ≥ 0.7
4. **Conflict Detection:** No contradiction with existing approved annotations

7.2 Continuous Learning

Rejected annotations feed back into the system:

- Rejection reasons stored for FM prompt refinement
- Modification diffs used to calibrate generation style
- Approval rates tracked per category for quality metrics

8 Expected Outcomes

8.1 Quantitative Benefits

Table 3: Projected Impact Metrics

Metric	Current	Projected
Active SME annotations	~10	100+
SME time per annotation	70 min	10 min
Coverage (compliance categories)	3/7	7/7
Evidence chain completeness	~60%	95%+
Annotation format consistency	Variable	Standardized

8.2 Qualitative Benefits

- **SME Focus Shift:** Experts focus on judgment, not documentation

- **Knowledge Capture:** Implicit expertise made explicit at scale
- **Institutional Memory:** Decisions preserved with full context
- **Continuous Improvement:** System learns from every review

9 Future Work

9.1 Active Learning Integration

Future versions could implement active learning to prioritize which divergence patterns to analyze:

- High-frequency patterns prioritized over rare cases
- Uncertainty sampling for ambiguous cases
- SME attention directed to highest-impact corrections

9.2 Multi-Modal Evidence

Extending evidence chains beyond RST to include:

- Code examples from production repositories
- Historical PR discussions and decisions
- Cross-references to related compliance frameworks

10 Conclusion

The FM-Assisted Annotation Generation framework represents a significant advancement in scaling human expertise capture for AI compliance systems. By analyzing divergence patterns between automated assessments and human judgments, foundation models can synthesize structured annotations that would otherwise require extensive SME effort to create manually.

This “Person-to-System Convolution” approach maintains human oversight through the review workflow while dramatically increasing annotation coverage and consistency. The framework integrates seamlessly with the existing Phase 2 Hybrid Architecture, requiring no changes to the ingestion, storage, or runtime components.

We anticipate this methodology will enable the MCP-RAG system to achieve comprehensive EE2 compliance coverage while reducing the burden on subject matter experts, ultimately improving the accuracy and reliability of automated compliance validation for the NOAA Global Workflow.

References

1. Phase 2 Hybrid Semantic Annotation Architecture Specification, NOAA EMC, November 2025
2. SDD Framework for AI-Assisted Development, NOAA EMC Technical Report, 2025
3. NOAA NWS EE2 Compliance Standards (standards.rst), National Weather Service
4. MCP Protocol Specification, Anthropic, 2024